Implementation and Evaluation of Branch Target Buffer Organizations for High-Throughput Instruction Fetch

Group 12

Archana Sindhu Sunil(436000717)
Dhanya Babu(536009727)
Sreeharine Govindaraj(137008873)

Table of Contents

1 **Introduction**
What is a BTB and why BTB organization matters for modern processors

2 **Motivation**
The problem and the goal of our project

3 **Project Roadmap**
An overview of the project methodology

4 **Background & Prior Work**
Overview of previous works that lead to BTB organization

5 **BTB Organizations**
I-BTB, R-BTB, B-BTB, MB-BTB — design implementation and tradeoffs

6 **Experimental Setup & Methodology**
Simulator, benchmarks, metrics

7 **Results and Evaluation**
IPC and MPKI across 13 SPEC CPU2017 benchmarks

8 **Conclusion & References**
Summary and citations

Introduction - Branch Target Buffer(BTB)

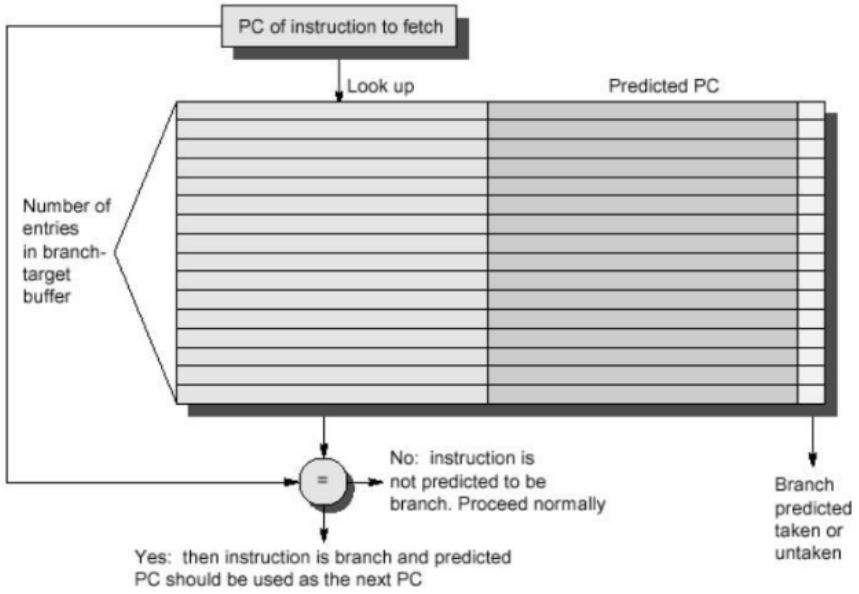
A **Branch Target Buffer (BTB) -** stores the target addresses of previously seen branch instructions, so the CPU can immediately know where to jump during the fetch stage.

Why do we need BTB?

When CPU fetches instructions, it does not yet know if instruction is a branch and where it will jump. Without BTB, CPU must wait until decode stage which causes stall. BTB solves this by giving the target early

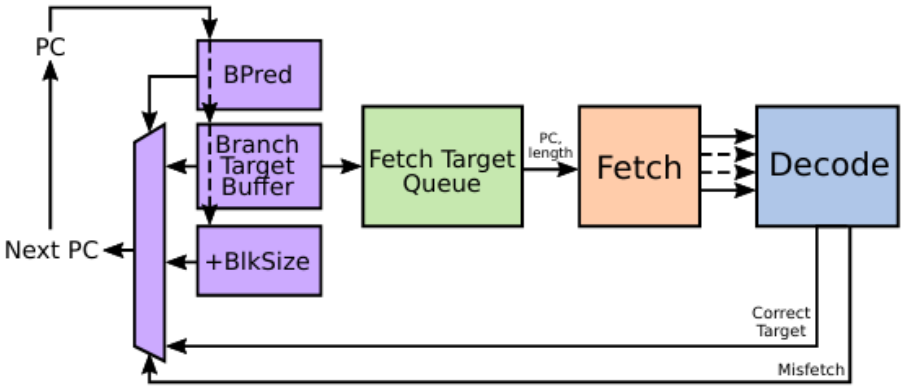
How BTB works ?

1) CPU fetches instruction using current PC
2) BTB is accessed using PC (like a cache lookup)
3) If BTB hit:
   It returns target address
   4) At the same time:
      Branch Predictor says taken / not taken
5) Next PC decision:
   - Taken → use BTB target
   - Not taken → use PC + 4



Only need to store Taken branches in the table

Introduction - Fetch Directed Prefetching (FDIP)



Decoupled PC generation flow using a BTB.

① Overlapping Cache Misses

PC Gen runs ahead of Fetch. Multiple I-Cache misses can be in-flight simultaneously, providing a natural instruction prefetching effect.

② Hides Taken Branch Penalty

If FTQ has backpressure (full ROB, narrow Fetch), the direct taken branch penalty is absorbed — pipeline doesn't stall waiting for next PC.

③ Fetch Across Taken Branches

Two FTQ entries mapping to different I-Cache interleaves can be consumed in one cycle, mitigating the opportunity cost of taken branches.

⚠ *Tradeoff: Decoupling lengthens the pipeline → increases misfetch and branch misprediction penalty.*

Motivation

THE PROBLEM

Working set is growing

Modern datacenter workloads average 138 KB instruction footprint. A BTB must cover this to avoid misses.

BTBs are getting very large

IBM Telum stores 270K+ branch targets. Intel and AMD ship multi-hundred-KB BTBs in production.

Bigger is not enough

Adding more entries increases area and power quadratically. At some point you cannot just keep scaling.

FDIP makes it worse

BTB must now prefetch ahead of fetch, not just keep up with it. It needs to cover even more of the working set.

The real question: *given a fixed area budget, how do you organize entries to maximize fetch PCs per cycle?*

Project Roadmap

Core idea
Evaluate how BTB organization (not just size) affects fetch throughput in wide, decoupled frontends.

Designs compared
Implement and evaluate 4 BTB organizations (I-BTB, R-BTB, B-BTB, MB-BTB) in ChampSim on SPEC CPU2017 benchmarks and compare against the paper's findings on CVP-1 ARMv8 traces.

What we measure
Arithmetic IPC, Geometric MPKI

Workloads and setup
Run server-style traces/SPEC CPU2017 with identical pipeline and predictor settings across all BTBs.

Goal: identify the BTB organization that best sustains instruction supply under realistic area and latency constraints.

Background & Prior Work

1984 — **Lee & Smith**
BTB origin — first branch target buffer proposal

1999 — **Reinman et al.**
FDIP origin — BTB-steered instruction cache prefetching

2013 — **Kaynak et al.**
Server workloads exposed the frontend bottleneck

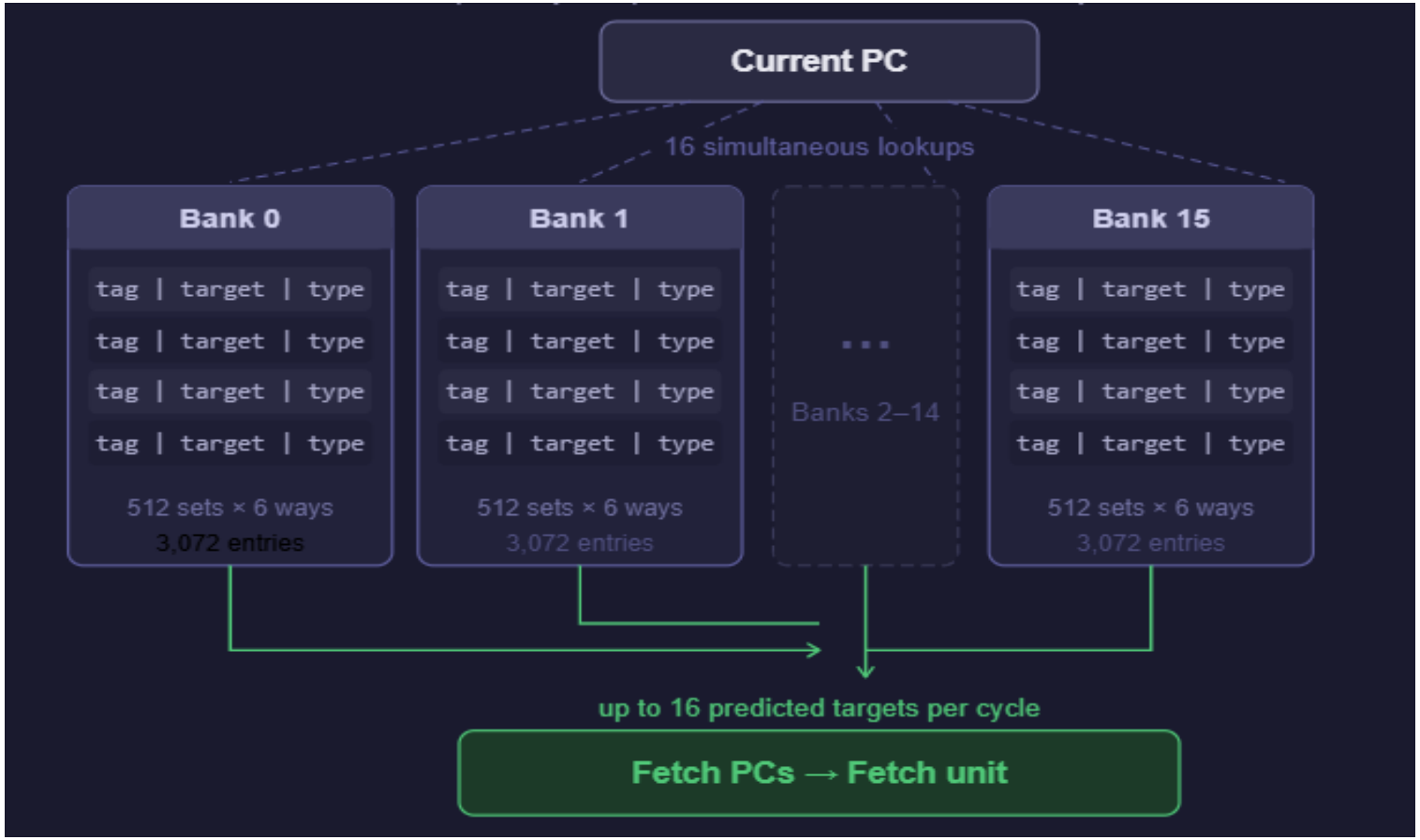
2013 — **Bonanno et al.**
Industry responded with two-level BTB hierarchies

2021 — **IBM Telum**
Ultimate industrial scale — 270K entries

2021 — **Ishii et al.**
Re-established FDIP with modern baselines

2023 — **Perais & Sheikh**
Systematic BTB organization comparison

Instruction BTB (I-BTB)



- Most storage-efficient per tracked branch, but interleave count must scale with fetch width — impractical for very wide-fetch designs
- Poorly suited to small first-level BTBs that require associative lookup

Key Properties

1 branch per entry
No storage sharing

16 banks (I-BTB 16)
Up to 16 fetch PCs/cycle

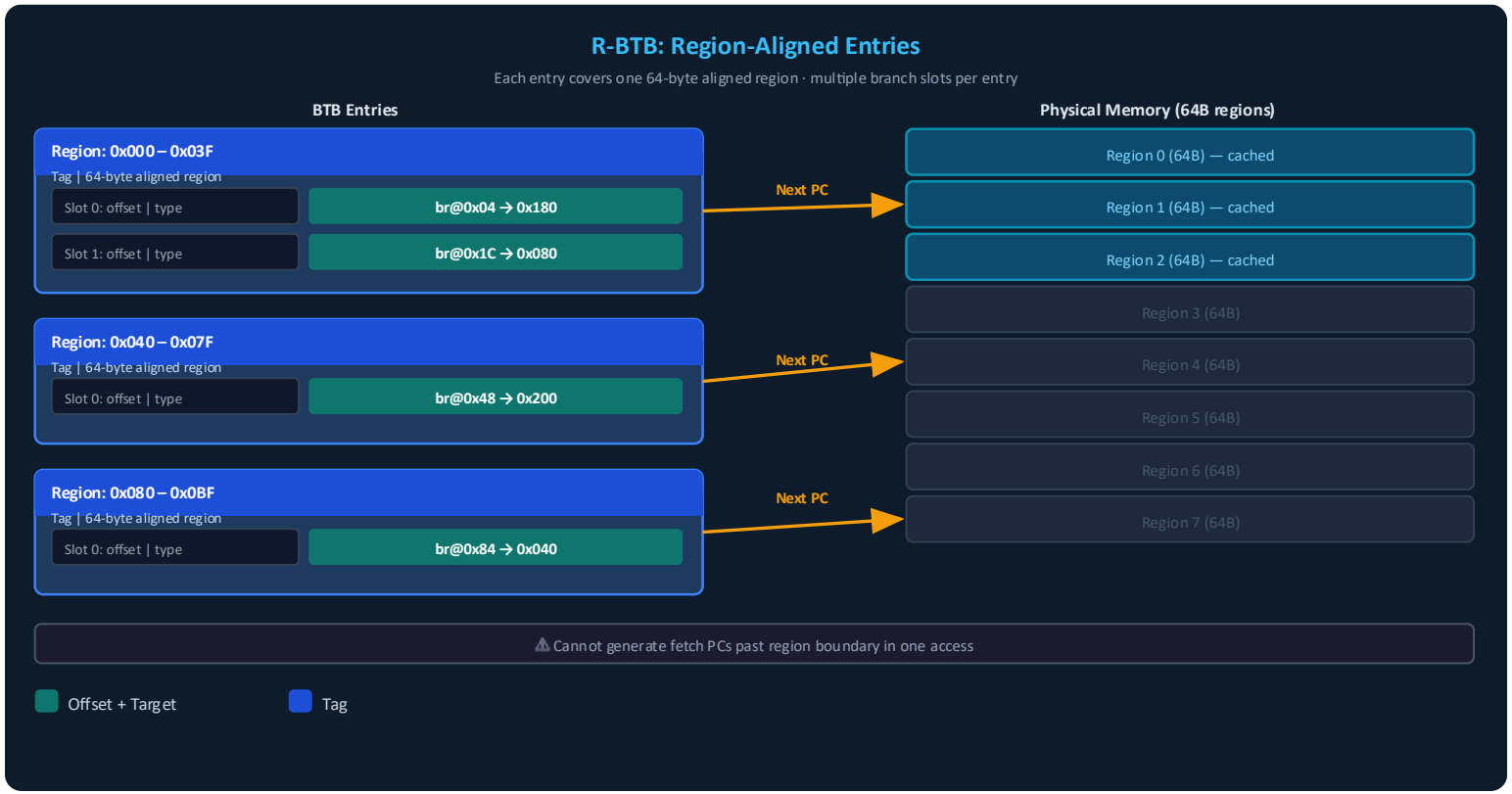
Zero redundancy
Each branch in ≤1 entry

Most agile eviction
New branch → evicts one

Never-taken filter
Conditionals not stored

Highest tag overhead
1 tag per branch target

Region BTB (R-BTB)



- R-BTB groups branches by 64B-aligned memory region.
- Each entry stores up to 3 branch targets.
- The key limitation - a single access cannot cross a region boundary to generate the next fetch PC.

Key Properties

64B region-aligned entries
All branches in a 64B window share one BTB entry

3 branch slots per entry
Each slot stores: offset, target, type, taken-hint

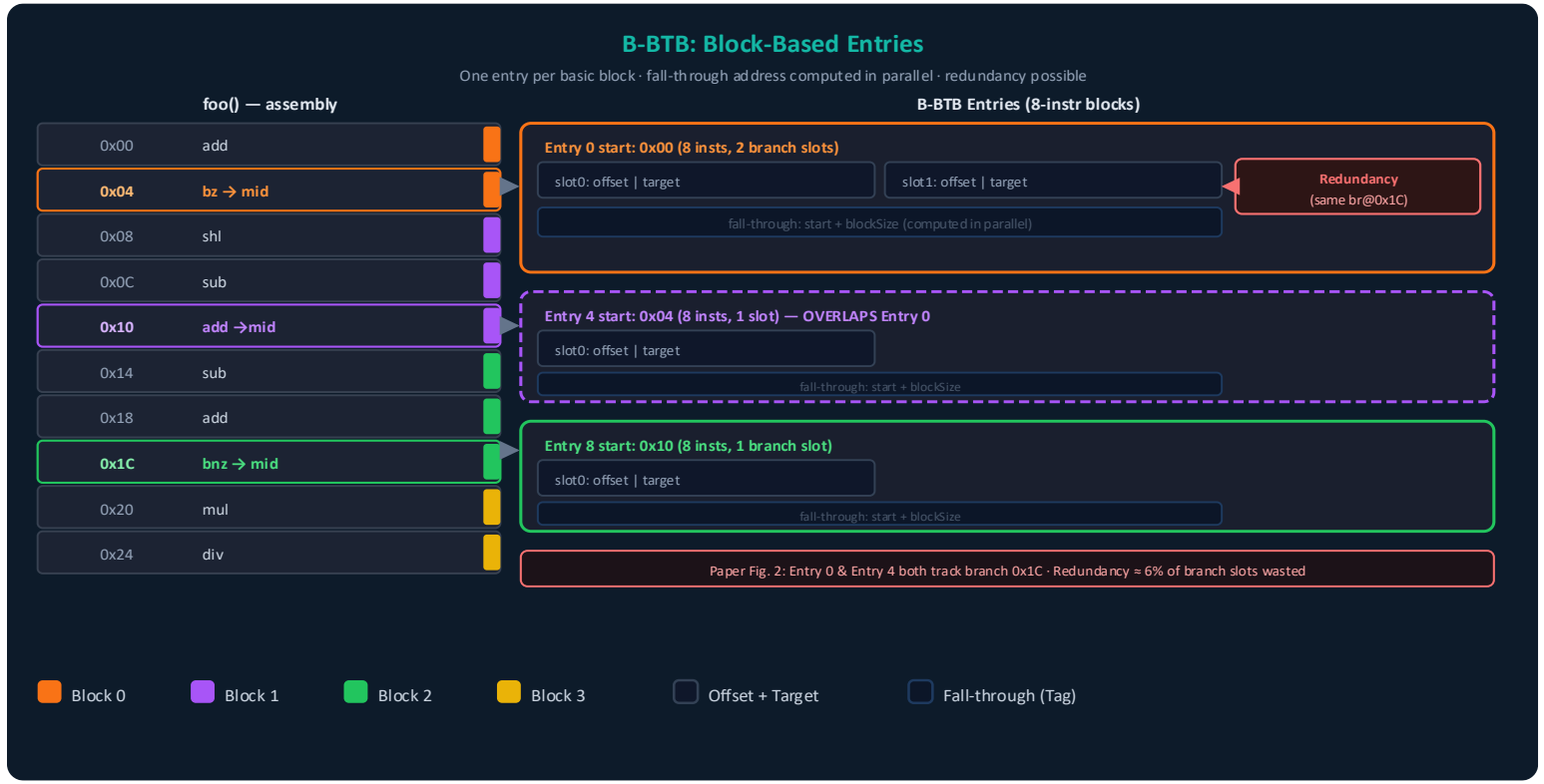
LRU replacement (slot and entry)
Evicts least-recently-used slot when entry is full

Region boundary limitation
Cannot fetch past a 64B boundary in single access

Two-level BTB hierarchy
L1 and L2

Multiple branches → one region tag
Lower tag cost and improved storage efficiency

Block BTB (B-BTB)



- B-BTB groups branches by basic block rather than fixed memory regions.
- Blocks are flexible and can cross address boundaries - single BTB access covers more instructions.
- The split mechanic prevents evictions when a block is full

Key Properties

Block-based entries
One entry per basic block

2 branch slots per entry
Each slot stores: branch PC, target, type, taken-hint

Redundancy (~6% storage waste)
Mid-block targets create overlapping "synonym" blocks

Entry splitting on overflow
New branch allocates a fresh entry instead of evicting

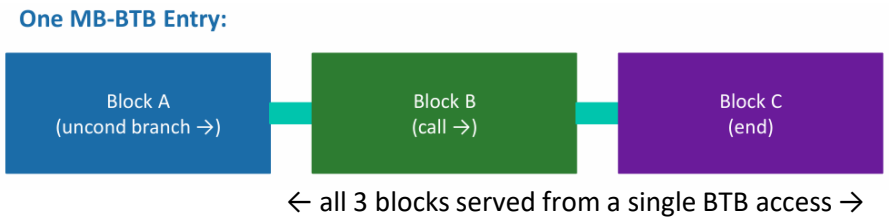
Two-level BTB hierarchy
L1 and L2

Can cross address boundaries
Variable-length blocks; not limited to fixed 64B regions

MultiBlock BTB (MB-BTB)

Core Idea

Extends B-BTB: when a slot ends with an unconditional branch, “pull” the target block into the same entry. One BTB access yields multiple basic blocks of fetch PCs.



Eligible Branch Types for Pulling Target Block

Unconditional Direct	Always eligible — immediate pull
Direct Calls	Eligible — large IPC gain (+16.5%)
Always-taken Conditional	Pull immediately on first seen taken
Stable Indirect Branch	After 63 same-target hits

- MB-BTB extends B-BTB by chaining successor blocks into a single entry, yielding multiple fetch PCs per access.
- Gains are highest when branch MPKI is high or the pipeline frequently refills after flushes.

Key Properties

Multi-block chaining
Up to 3 blocks per BTB access

2 branch slots per entry
Inherited from B-BTB; slot adds follow + stabl_ctr

Entry splitting on overflow
Split instead of evicting metadata

Last-slot pull restriction
Reduces redundancy from shared call targets

Two-level BTB hierarchy
L1 and L2

Inherits ~6% redundancy
Mitigated by last-slot restriction

BTB Organizations — Comparison

I-BTB (Instruction BTB)

1 br/entry

One entry per branch instruction. Keyed by branch IP. Multiple banks accessed in parallel for wide fetch.

✓ Zero redundancy. Most agile eviction. Simple.

✗ High tag overhead. Needs 16 banks for 16-wide fetch.

R-BTB (Region BTB)

n br/region

Entries keyed by 64B-aligned region. All branches in same 64B region share one entry with n branch slots.

✓ Amortized tag overhead. Efficient for dense regions.

✗ Cannot cross region boundary in one access.

B-BTB (Block BTB)

block/entry

Entries keyed by block start PC. A block = all instructions until the next taken branch. Variable-length.

✓ Single access covers full block. No region limit.

✗ ~6% redundancy from synonym blocks.

MB-BTB (MultiBlock BTB)

multi-block

Extends B-BTB: when slot[0] has follow=1, the target block is pulled inline into slot[1]. One access → 2 blocks.

✓ Hides taken-branch stall. Better storage density.

✗ L1 contention limits IPC gains.

Paper uses geomean IPC as primary metric across 147 CVP-1 ARMv8 server traces
All configs use same hierarchy: L1=3K entries, L2=13K entries

Experimental Setup & Methodology

Simulator
ChampSim (trace-based, cycle-level) with decoupled frontend
Custom BTB modules: I-BTB, R-BTB, B-BTB, MB-BTB.
Built with hashed perceptron + per-config BTB selection.

Workloads — SPEC CPU 2017
13 SPECspeed 2017 benchmarks (Integer & Floating Point):
• 602.gcc_s • 603.bwaves_s • 605.mcf_s • 620.omnetpp_s
• 623.xalancbmk_s • 625.x264_s • 627.cam4_s
• 631.deepsjeng_s • 638.imagick_s • 641.leela_s • 644.nab_s
• 648.exchange2_s • 654.roms_s

Fair Comparison
Same core/memory setup and run length for all BTB organizations.
Fixed BTB hierarchy: L1 = 512x6, L2 = 1024x13

Evaluation Metric
Primary metric: IPC across 13 traces.
Also reported: branch prediction accuracy and branch MPKI.

Pipeline Configuration

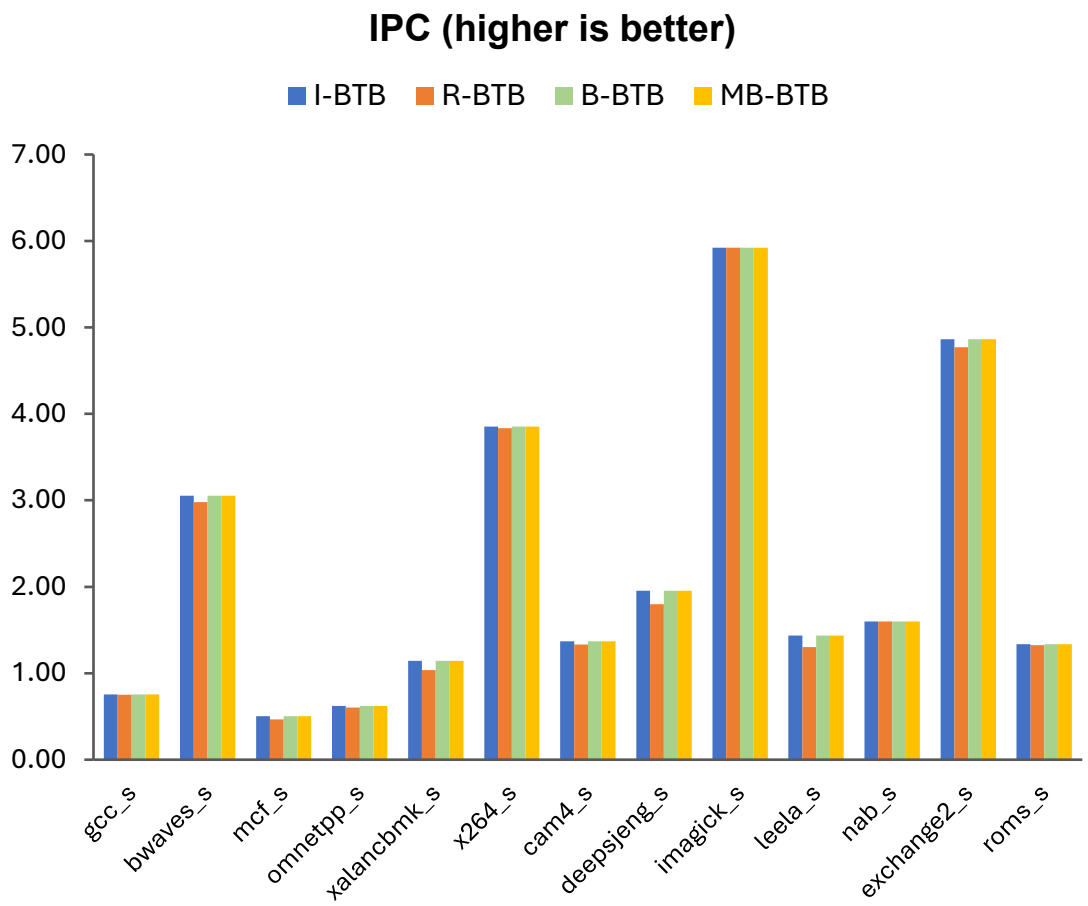
Branch Predictor	Hashed Perceptron Predictor
RAS	64-entry Return Address Stack
L1 BTB	3K-entry (512×6 ways), 0-cycle penalty
L2 BTB	13K-entry (1024×13 ways), 3-cycle penalty
Fetch Width	16-wide frontend - fetch/decode/dispatch/retire
ROB	352 entries
L1 I-Cache	32KB (64 sets × 8 ways), 4-cycle latency
LLC	2MB, 20-cycle latency
DRAM	DRAM: 4 GiB modeled memory, 3200 MT/s, single-channel

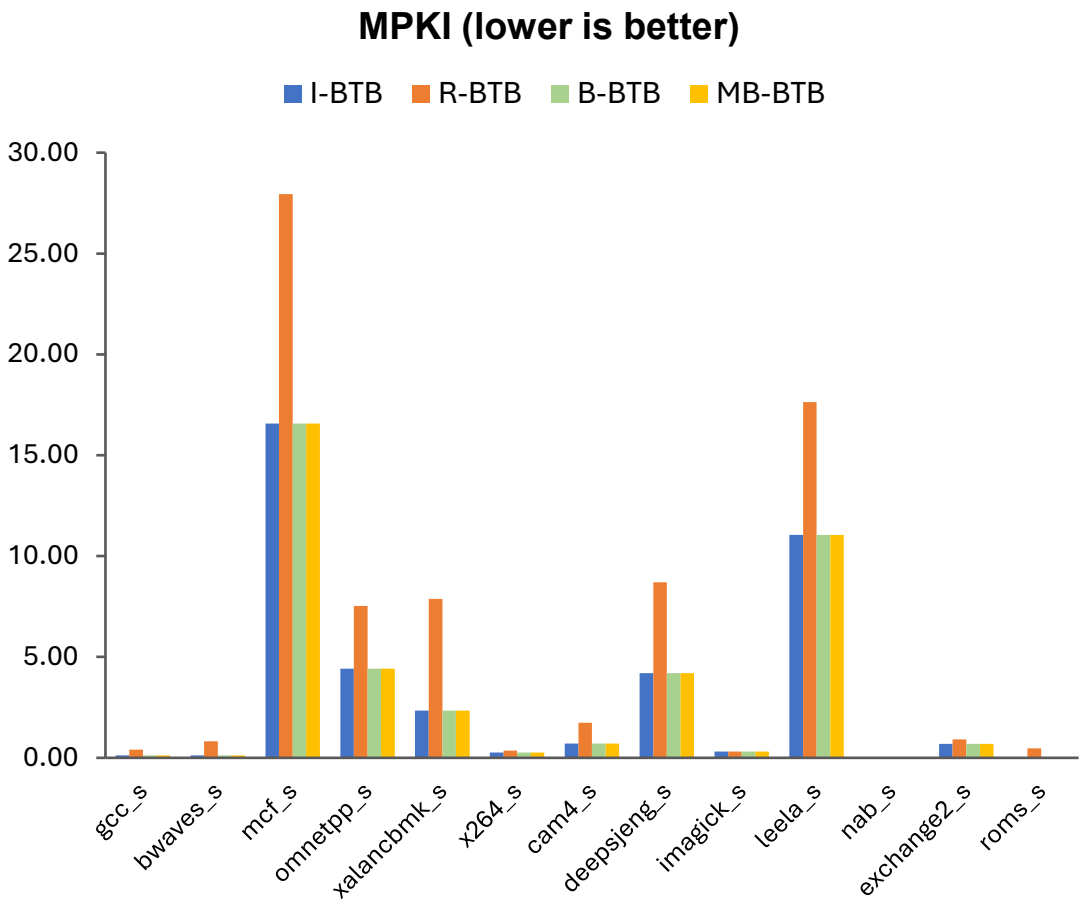
Results — IPC & MPKI

	IPC (higher is better)				MPKI (lower is better)			
Benchmark	I-BTB	R-BTB	B-BTB	MB-BTB	I-BTB	R-BTB	B-BTB	MB-BTB
602.gcc_s	0.7539	0.752707	0.7539	0.7539	0.116	0.400913	0.116	0.116
603.bwaves_s	3.0518	2.9795	3.0518	3.0518	0.117	0.823341	0.117	0.117
605.mcf_s	0.5032	0.467554	0.5032	0.5032	16.57	27.9506	16.57	16.57
620.omnetpp_s	0.6233	0.604115	0.6233	0.6233	4.423	7.52637	4.423	4.423
623.xalancbmk_s	1.1423	1.0357	1.1421	1.1421	2.338	7.87171	2.338	2.338
625.x264_s	3.8533	3.8329	3.8535	3.8535	0.261	0.352185	0.261	0.261
627.cam4_s	1.3678	1.33295	1.3678	1.3678	0.711	1.73465	0.711	0.711
631.deepsjeng_s	1.9535	1.79847	1.9535	1.9535	4.193	8.70923	4.193	4.193
638.imagick_s	5.9208	5.92083	5.9208	5.9208	0.313	0.312792	0.313	0.313
641.leela_s	1.4340	1.30157	1.4340	1.4340	11.05	17.637	11.05	11.05
644.nab_s	1.6004	1.60031	1.6004	1.6004	0.018	0.020005	0.018	0.018
648.exchange2_s	4.8638	4.76945	4.8638	4.8638	0.686	0.912989	0.686	0.686
654.roms_s	1.3356	1.32458	1.3356	1.3356	0.045	0.475732	0.045	0.045

Arith Mean IPC — I-BTB:2.185 R-BTB:1.980 B-BTB:2.185 MB-BTB:2.185 | Geomean MPKI — I-BTB:0.646 R-BTB:1.4620 B-BTB:0.646 MB-BTB:0.646

Results — IPC & MPKI





Arith Mean IPC: I-BTB 2.185 | R-BTB 1.980 | B-BTB 2.185 | MB-BTB 2.185
Geomean MPKI: I-BTB 0.646 | R-BTB 1.4620 | B-BTB 0.646 | MB-BTB 0.646

Evaluation

Paper Results (CVP-1 ARMv8)

I-BTB (Instruction BTB)
Geomean IPC 1.79. Highest of all configs. L1 hit rate 76.3%, MPKI 0.84.
Baseline reference for all comparisons.

R-BTB 3BS (best R-BTB config)
Geomean IPC ~1.76.
Mainly due to region-boundary and slot pressure limits.

B-BTB 1BS + Entry Splitting
Geomean IPC 1.78. Splitting adds +2.6% over B-BTB 1BS baseline.
~6% redundancy. Best practical design per paper.

MB-BTB 2BS AllBr
Geomean IPC 1.74 (constrained). Fetch PCs/access significantly higher.
Gains grow in higher-stress conditions (e.g., ideal backend or higher branch MPKI).

Our Results (SPEC CPU2017)

IPC: I-BTB = B-BTB = MB-BTB > R-BTB
Arith Mean IPC: I/B/MB-BTB = 2.185, R-BTB = 1.980.
R-BTB paid a consistent 9.4% IPC penalty across all 13 benchmarks, validating the paper's region-boundary bottleneck finding.

MPKI: R-BTB 2.3x higher than others
Geomean MPKI: R-BTB 1.462 vs 0.646 for others.
Worst case: 605.mcf_s (pointer-chasing) hit R-BTB MPKI of 27.95 vs 16.57 for I-BTB, confirming slot eviction losses under pressure.

ISA and Workload Gap
Paper: 147 CVP-1 ARMv8 server traces (avg 138 KB footprint).
Ours: 13 x86 SPEC benchmarks (smaller footprints).
B-BTB and MB-BTB IPC gains over I-BTB narrowed due to lower instruction-cache pressure.

MB-BTB: benefit hidden by low MPKI
MB-BTB's multiblock chaining advantage only materializes when pipeline refills are frequent. SPEC's 0.646 geomean branch MPKI was insufficient to expose that benefit vs the paper's server traces.

Key Takeaway: R-BTB consistently underperforms due to region boundary constraints. B-BTB (1BS+Split) and MB-BTB match I-BTB on SPEC, confirming that BTB organization drives fetch throughput more than raw capacity.

Conclusion

WHAT WE DID

Implemented All 4 BTB Organizations

Built I-BTB, R-BTB, B-BTB, and MB-BTB in ChampSim. Each follows the paper's structural definitions with matching L1/L2 hierarchy and realistic entry counts.

Evaluated on 13 SPEC CPU2017 Benchmarks

Measured IPC and MPKI across 602.gcc through 654.roms. Same ranking order as Perais & Sheikh CVP-1 ARMv8 findings — validated on x86 workloads.

Identified Structural Bottlenecks

R-BTB's region boundary constraint causes 9.4% mean IPC loss and 2.3x higher MPKI. Slot contention in 64B regions — not BTB size — is the dominant bottleneck.

Validated B-BTB as Optimal Practical Design

B-BTB and MB-BTB match I-BTB IPC on all 13 benchmarks. MB-BTB gains emerge at high branch MPKI and deep ILP.

Key Finding

BTB organization — not capacity — is the primary driver of fetch throughput. B-BTB with entry splitting is the best practical tradeoff; MB-BTB closes the gap when branch MPKI is high.

Future scope

01

Heterogeneous BTB Hierarchies
Mix B-BTB at L1 with R-BTB at L2/L3 to balance low-latency fetch and storage efficiency across levels.

02

Adaptive MB-BTB Eligibility
Develop smarter, workload-aware thresholds for which conditional and indirect branches qualify for multi-block chaining.

03

ML-Based Branch Slot Allocation
Paper uses fixed heuristics (e.g., 63-consecutive threshold for indirect branches). A learned predictor could dynamically decide which branches get slots and which blocks get chained

04

Density + Organization Co-design
Combine compression (PDede, BTB-X, tag indirection) with MB-BTB block chaining for storage-efficient high-throughput designs.

References

1. A. Perais and B. R. Sheikh, "Branch Target Buffer Organizations," in Proceedings of the 56th IEEE/ACM International Symposium on Microarchitecture (MICRO), Toronto, Canada, Oct. 2023. DOI: 10.1145/3613424.3614258
2. C. Lee and J. E. Smith, "Branch prediction strategies and branch target buffer design," IEEE Computer, vol. 17, no. 1, pp. 6–22, Jan. 1984.
3. J. Bonanno, A. Collura, D. Lipetz, U. Mayer, B. Prasky and A. Saporito, "Two level bulk preload branch prediction," 2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA), Shenzhen, China, 2013, pp. 71-82, doi: 10.1109/HPCA.2013.6522308.
4. D. A. Jimenez, "Multiperspective perceptron predictor with TAGE," in 5th Championship Branch Prediction (CBP-5), Seoul, Korea, 2016.
5. G. Reinman, B. Calder, and T. Austin, "Fetch Directed Instruction Prefetching," in Proceedings of the 32nd Annual ACM/IEEE International Symposium on Microarchitecture (MICRO-32), Haifa, Israel, pp. 16–27, November 1999.
6. G. Reinman, T. Austin, and B. Calder, "A scalable front-end architecture for fast instruction delivery," in Proceedings of the 26th Annual International Symposium on Computer Architecture (ISCA), Atlanta, GA, May 1999, pp. 234–245.
7. C. Jacobi, "Real-time AI for Enterprise Workloads: the IBM Telum Processor," in IEEE Hot Chips 33 Symposium (HCS), Palo Alto, CA, USA, Aug. 2021, pp. 1–22.
8. C. Kaynak, B. Grot and B. Falsafi, "SHIFT: Shared history instruction fetch for lean-core server processors," 2013 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), Davis, CA, USA, 2013, pp. 272-283.
9. ChampSim Contributors, ChampSim: A Trace-Based Microarchitecture Simulator, commit 29f568c. [Online]. Available: https://github.com/ChampSim/ChampSim
10. Standard Performance Evaluation Corporation, SPEC CPU2017 Benchmark Suite, 2017. [Online]. Available: https://www.spec.org/cpu2017